

Rapid terrain adaptation through continual learning of central pattern generator parameters for quadrupedal robot locomotion

Alex Pitoiset¹, Tim N. Nisslbeck¹, Ajith Anil Meera¹ and Wouter M. Kouw¹

Abstract—

I. INTRODUCTION

Achieving the same level of adaptability as animals is crucial for deploying legged machines in constantly changing real world environments. To bridge the gap between engineering and nature, engineers have frequently turned to biology for ideas, replicating animal morphologies. For example, snake-like robots for constrained environments or quadruped platforms for irregular terrain [8], [3]. These morphologies inherit the mechanical advantages of their biological counterparts. However, their control systems often lack in adaptability. For example, the Boston Dynamics quadrupedal robot, Spot, mimics animal legs. However, it relies on centralized and computationally intensive controllers that require multiple sensors [3]. Although this approach performs well, it remains complex and expensive, limiting its scalability for wider deployment and access.

To address these challenges, a promising approach to improve robotic locomotion is to use Central Pattern Generators (CPGs), which are neural circuits that generate high-dimensional rhythmic signals. In vertebrates, CPGs are located in the spinal cord and produce periodic coordinated movements such as walking and breathing [5]. By automating these repetitive physical patterns, CPGs allow the brain to redirect its resources to other cognitive tasks, such as selecting safe paths or responding to unexpected stimuli. These systems are triggered by simple inputs from higher brain centers. The brain can modulate their rhythmic output to control parameters such as locomotion speed and gait. Moreover, CPGs remain coupled to the body and environment through sensory feedback [2]. This decentralized architecture reduces the delays for commands and the required bandwidth while simplifying high-level control [6]. These features make CPG-based controllers an attractive solution as they are less energy intensive, simpler, and more affordable.

A. Related Work

Inspired by biological principles, various CPG models have been implemented in robotics. Among them, coupled nonlinear oscillators have emerged as biologically plausible and computationally efficient for generating rhythmic movement [1], [6].

The power of coupled nonlinear oscillators lies in their ability to inhibit each other. This coupling behavior enables synchronization between joints, producing coordinated gaits and smooth transitions between them [1], [10], [?]. Such mechanisms mimic biological inter-neuronal inhibition and allow the generation of out-of-phase rhythmic signals. They also exhibit a limit-cycle behavior, producing periodic and self-sustaining oscillations that quickly recover from transient perturbations [10]. In addition, their compact dynamics, governed by a small set of parameters (e.g. frequency and amplitude), makes them appropriate for embedded systems with limited resources. Despite their promise, most existing implementations target high-end robotic platforms. Their applicability to compact, affordable systems remains under-explored, which underpins the present study.

In the following, we contribute to the field with

- the design of an intelligent agent that continuously updates parameters of a central pattern generator, and
- by benchmarking this agent against offline grid search and online Bayesian optimization.

II. PROBLEM STATEMENT

a) *Locomotion task*: Consider a quadruped robot navigating terrain with condition τ (characterised, for instance, by surface friction μ_s or slope angle ϕ). The robot has four legs, each with two joints driven by servomotors: a hip angle $\phi_i^h(t)$ and a knee angle $\phi_i^k(t)$ for leg $i = 1, \dots, 4$. At discrete time step k with sampling interval Δt , the base pose is described by its position $\mathbf{q}_k = (q_k^x, q_k^y, q_k^z)$ and Euler angles $\gamma_k = (\gamma_k^p, \gamma_k^r, \gamma_k^y)$ for pitch, roll, and yaw, respectively. We measure locomotion quality by the scalar objective $J(\theta, \tau)$ (defined in Sec. IV), which depends on both the CPG parameter vector $\theta \in \mathbb{R}^8$ and the terrain τ . A fall is declared if $q_k^z < h_{\min}$ or $|\gamma_k^r| > \gamma_{\max}$ at any time step, where $h_{\min}$ and $\gamma_{\max}$ are safety thresholds.

b) *Terrain-dependent optimal parameters*: For a given terrain τ there exists an optimal CPG parameter vector

$$\theta^*(\tau) = \arg \max_{\theta} \mathbb{E}[J(\theta, \tau)], \quad (1)$$

where the expectation is over the stochastic outcomes of a locomotion trial. Critically, the performance landscape $J(\theta, \tau)$ shifts when the terrain changes: parameters tuned for flat ground become suboptimal on a slope or a slippery surface (Fig. 1a). A terrain change $\tau \rightarrow \tau'$ therefore immediately induces a *parameter mismatch*

$$\delta_k = \theta_k - \theta^*(\tau'), \quad (2)$$

*This work was supported by the Eindhoven Artificial Intelligence Systems Institute

¹Department of Electrical Engineering, Eindhoven University of Technology, Netherlands w.m.kouw@tue.nl

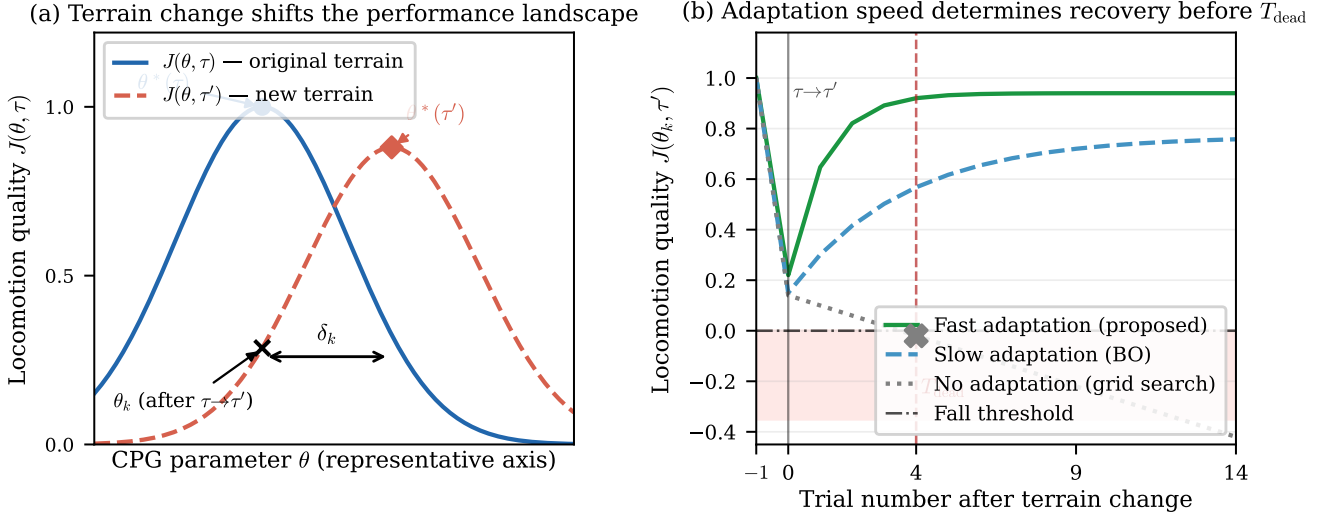


Fig. 1. Terrain change creates adaptation urgency (schematic). **(a)** The performance landscape $J(\theta, \tau)$ shifts when the terrain changes from τ (solid) to τ' (dashed). Parameters optimal for τ lie in a suboptimal region for τ' , creating a parameter mismatch δ_k . **(b)** Without rapid adaptation the mismatch initiates a positive feedback loop that degrades locomotion quality below the fall threshold before the adaptation deadline T_{dead} . A fast continual learning agent (green, solid) converges within a few trials; a slow optimizer (blue, dashed) does not recover in time; an agent with fixed parameters (grey, dotted) degrades monotonically.

between the robot's current parameters θ_k and the new terrain optimum.

c) The positive feedback cycle: The mismatch δ_k does not merely inflict a static performance penalty—it initiates a positive feedback loop that, if left uncontrolled, accelerates toward an irreversible fall (Fig. 1b):

- 1) *Mismatch $\rightarrow$ gait error.* Suboptimal CPG parameters produce joint trajectories that deviate from the nominal limit-cycle shape, generating a body-pose error e_k .
- 2) *Gait error $\rightarrow$ body perturbation.* Imperfect foot placement excites roll and pitch deviations in γ_k , which grow with the magnitude of the deviation.
- 3) *Body perturbation $\rightarrow$ CPG disturbance.* Large orientation excursions corrupt the sensory feedback driving the oscillator network and pull the CPG state off its limit-cycle attractor.
- 4) *CPG disturbance $\rightarrow$ larger gait error.* As the CPG drifts from its attractor, joint trajectories become increasingly erratic, amplifying e_k and closing the loop.

The escalation rate grows with $|\delta_k|$, so that a recoverable suboptimality can quickly become an irreversible fall. This introduces an *adaptation deadline* T_{dead} : the number of trials within which the agent must drive $\delta_k \approx 0$ before the fall thresholds h_{min} , γ_{max} are breached.

d) Design criterion: The CPG controller must exhibit stable limit-cycle behaviour, allow independent stance and swing modulation [2], and expose a low-dimensional, continuously adjustable parameter vector θ that can be updated online after each locomotion trial. Most importantly, the adaptation agent must converge to $\theta^*(\tau')$ within $O(\text{few})$ trials—before the positive feedback cycle leads to a fall.

III. METHODS

A. Central Pattern Generator

The CPGs are modeled as a network of four mutually coupled Hopf oscillators, each driving one limb: left front (LF), right front (RF), left hind (LH), and right hind (RH). Coordination between limbs arises from inhibitory coupling through a gait-specific coupling matrix $K = [k_{ij}]$. Each oscillator evolves as:

$$\dot{x}_i = \alpha(u - r_i^2)x_i - \omega_i y_i, \quad (3)$$

$$\dot{y}_i = \beta(u - r_i^2)y_i + \omega_i x_i + \gamma \sum_{j=1}^4 k_{ij} y_j, \quad (4)$$

$$\omega_i = \frac{\omega_{\text{stance}}}{1 + e^{-by_i}} + \frac{\omega_{\text{swing}}}{1 + e^{by_i}}, \quad (5)$$

where $r_i = \sqrt{x_i^2 + y_i^2}$, u sets the squared limit-cycle amplitude, $\alpha, \beta > 0$ are convergence rates, γ is the coupling gain, and b controls the sharpness of the frequency transition between stance and swing phases.

B. Mapping to Joint Angles

The CPG outputs (x_i, y_i) are mapped to hip and knee joint angles:

$$\theta_{\text{hip}} = \theta_{\text{hip base}} + \theta_{\text{hip range}} \cdot x_i, \quad (6)$$

$$\theta_{\text{knee}} = \theta_{\text{knee base}} - \theta_{\text{knee range}} \cdot \max(0, y_i), \quad (7)$$

where x_i drives forward-backward hip swing and y_i controls knee flexion; negative y_i is suppressed to prevent hyperextension.

C. Autoregressive Active Inference Agent

The agent is based on a multivariate autoregressive model with exogenous inputs (MARX) [7]. It maintains a belief over dynamics parameters updated from observations, and selects controls by minimising an Expected Free Energy (EFE) objective over a planning horizon.

1) *Model*: The likelihood is a Gaussian autoregression with coefficient matrix $A \in \mathbb{R}^{D_x \times D_y}$ and noise precision $W \in \mathbb{R}_+^{D_y \times D_y}$. Denoting $\Theta = (A, W)$:

$$p(y_k | \Theta, u_k, \bar{u}_k, \bar{y}_k) = \mathcal{N}(y_k | A^\top x_k, W^{-1}), \quad (8)$$

where $\bar{u}_k$ and $\bar{y}_k$ are running memories of the M_u previous inputs and M_y previous outputs, and the regressor concatenates the current control with the memories:

$$x_k = \begin{bmatrix} u_k \\ \bar{u}_k \\ \bar{y}_k \end{bmatrix}, \quad D_x = D_u(M_u + 1) + D_y M_y. \quad (9)$$

The prior over parameters is a matrix-normal-Wishart (MNW) distribution [9, ID: P323]:

$$p(\Theta) = \mathcal{M}\mathcal{N}(A | M_0, \Lambda_0^{-1}, W^{-1}) \mathcal{W}(W | \Omega_0^{-1}, \nu_0). \quad (10)$$

Controls are given an independent Gaussian prior for regularisation, with precision Υ :

$$p(u_k) = \mathcal{N}(u_k | 0, \Upsilon^{-1}). \quad (11)$$

The target observation is encoded as a Gaussian goal prior:

$$p(y_t | y_*) = \mathcal{N}(y_t | m_*, S_*). \quad (12)$$

2) *Parameter Estimation*: Because the MNW distribution is conjugate to the Gaussian likelihood, Bayesian filtering yields a closed-form MNW posterior $p(\Theta | \mathcal{D}_k) = \mathcal{M}\mathcal{N}\mathcal{W}(A, W | M_k, \Lambda_k^{-1}, \Omega_k^{-1}, \nu_k)$ at each step k , where $\mathcal{D}_k = \{y_i, u_i\}_{i=1}^k$. The parameters are updated by rank-one corrections:

$$\nu_k = \nu_{k-1} + 1, \quad (13)$$

$$\Lambda_k = \Lambda_{k-1} + x_k x_k^\top, \quad (14)$$

$$M_k = (\Lambda_{k-1} + x_k x_k^\top)^{-1} (\Lambda_{k-1} M_{k-1} + x_k y_k^\top), \quad (15)$$

$$\begin{aligned} \Omega_k &= \Omega_{k-1} + y_k y_k^\top + M_{k-1}^\top \Lambda_{k-1} M_{k-1} \\ &\quad - (\Lambda_{k-1} M_{k-1} + x_k y_k^\top)^\top (\Lambda_{k-1} + x_k x_k^\top)^{-1} (\Lambda_{k-1} M_{k-1} + x_k y_k^\top). \end{aligned}$$

Marginalising the likelihood over the MNW prior yields a multivariate Student-t posterior predictive:

$$p(y_k | u_k, \mathcal{D}_{k-1}) = \mathcal{T}_{\eta_k}(y_k | \mu_k(u_k), \Sigma_k(u_k)), \quad (16)$$

with $\eta_k = \nu_{k-1} - D_y + 1$ and

$$\mu_k(u_k) = M_{k-1}^\top x_k, \quad (17)$$

$$\Sigma_k(u_k) = \frac{\Omega_{k-1}}{\nu_{k-1} - D_y + 1} \left(1 + x_k^\top \Lambda_{k-1}^{-1} x_k \right). \quad (18)$$

Both the mean μ_k and scale Σ_k are smooth functions of u_k .

3) *Planning*: To select controls over a horizon $\tau = (k+1, \dots, k+T)$, the agent unrolls the autoregressive model forward. At each step $t \in \tau$, the regressor is formed as $x_t = [u_t, \bar{u}_t, \hat{y}_{t-1}]^\top$, where future outputs not yet observed are substituted with MAP estimates of earlier predictives, $\hat{y}_{t-1} = [\mu_{t-1}(u_{t-1}), \mu_{t-2}(u_{t-2}), \dots]$. This yields a sequence of Student-t predictives $p(y_t | u_t, \mathcal{D}_k)$ with means $\mu_t(u_t)$ and scales $\Sigma_t(u_t)$.

The planning objective is the Expected Free Energy (EFE), defined as the sum of KL divergences between each posterior predictive and the goal prior:

$$\mathcal{J}(u_\tau) = \sum_{t \in \tau} D_{\text{KL}}[p(y_t | u_t, \mathcal{D}_k) \parallel p(y_t | y_*)]. \quad (19)$$

Expanding this KL divergence yields goal-seeking terms (functions of μ_t , penalising deviation from m_*) and information-seeking terms (functions of Σ_t , penalising output uncertainty). Together with the control prior, these produce the MAP action selection problem over the planning horizon:

$$\begin{aligned} \hat{u}_\tau = \arg \min_{u_\tau \in \mathbb{U}^T} \sum_{t \in \tau} & \left[-\frac{1}{2} \ln |\Sigma_t(u_t)| + \frac{1}{2} \text{tr} \left[S_*^{-1} \frac{\eta_t}{\eta_t - 2} \Sigma_t(u_t) \right] \right. \\ & \left. + \frac{1}{2} (\mu_t(u_t) - m_*)^\top S_*^{-1} (\mu_t(u_t) - m_*) + u_t^\top \Upsilon u_t \right], \quad (20) \end{aligned}$$

where $\mathbb{U}^T = \mathbb{U} \times \dots \times \mathbb{U}$ is the space of affordable controls over the horizon.

IV. EXPERIMENTS

We evaluate three parameter-selection strategies for the Hopf-oscillator CPG controller on the Laikago quadruped in PyBullet: (i) an offline *grid search* over a small set of CPG parameters; (ii) online *Bayesian optimization* (BO) with a Gaussian-process surrogate and a UCB acquisition function; and (iii) the proposed *MARX-EFE* active inference agent, which continually updates CPG parameters by minimising expected free energy. Grid search serves as a fixed, non-adaptive reference point, BO serves as a state-of-the-art black-box optimizer, and MARX-EFE is the proposed continual learning agent.

All three methods optimize in the same 8-dimensional parameter space with identical bounds:

$$\theta = [\gamma, \omega_{\text{swing}}, \omega_{\text{stance}}, F_{\text{fast}}, K_{\text{stop}}, A_{\text{hip}}, A_{\text{knee}}, b],$$

corresponding to the coupling gain, the two phase-dependent frequencies, the Righetti-style fast-contact gain, the stop-gain, the hip and knee amplitudes, and the sigmoid sharpness b from Eq. 5. Each trial is a $T = 4.5$ s episode of simulated walking on flat ground with $\Delta t = 0.01$ s, a 1.5 s settling window excluded from the metrics, and a target forward velocity of 1.0 m/s. Parameter updates occur *online*: every trial carries the CPG state forward and smoothly interpolates between the previous and new parameters over the first 150 steps.

TABLE I

ONLINE CONTINUOUS RUN ACROSS VARYING FRICTION ($N = 50$ TERRAINS, 45 s, MEAN $\pm$ S.D.). LOWER IS BETTER EXCEPT DISTANCE.

Metric	BO (4.5 s, TR)	MARX-EFE (1 s)
Fall rate	54%	12%
Velocity error [m/s]	0.61 ± 0.36	0.58 ± 0.37
Tip-over [deg]	7.8 ± 1.9	7.3 ± 1.2
Yaw drift [deg]	50.6	44.7
Distance [m]	25.5	34.3

a) *Performance metrics:* We score each method on four quantities computed over the 3.0 s steady-state window: the scalar objective

$$J = w_1 R_v - w_2 \text{CoT} - w_3 \mathcal{S}, \quad (21)$$

with velocity reward $R_v = \frac{1}{T} \sum_k \exp(-\|v_k - v^*\|^2 / \sigma_v^2)$, cost of transport $\text{CoT} = \int P_{\text{mech}} dt / (mgd)$, and combined RMS stability $\mathcal{S} = \sqrt{\text{RMS}(\gamma^r)^2 + \text{RMS}(\gamma^p)^2}$; the forward distance d_y ; the lateral drift $|d_x|$; and the fall rate. To compare sample efficiency, we report the number of trials N_{walk} required to reach $J \geq J_{\text{th}}$ with cumulative fall rate below r_{th} , and the cumulative forward distance D_{cum} accumulated over the optimization horizon.

A. Online adaptation across friction transitions

Beyond the episodic optimisation above, we evaluate a fully *online* setting in which the terrain changes underfoot during a single uninterrupted bout, isolating each method’s *update rate*. The robot walks forward for $T = 45$ s (4500 control steps) with no resets, over flat ground whose lateral friction varies in space: a random sequence of zones (slippery $\mu=0.45$, normal 0.7, grip 1.1, rubber 1.6; 1.5–3.5 m long) is sampled per terrain and the ground friction is set each step from the robot’s forward position. The controller must hold the target velocity $v^* = 1.0$ m/s while repeatedly crossing surface changes it cannot reset away from.

The two adaptive methods update at different cadences. Online BO re-optimises once every 4.5 s: it scores the just-finished window with the objective of Eq. 21, augments its GP, and proposes the next parameters by maximising UCB within a TuRBO-style trust region [4] of radius $\rho = 0.2$ around the incumbent, interpolating over the next 1.5 s. The trust region limits per-update jumps and equalises caution between the methods. MARX-EFE updates its posterior every step ($\Delta t = 0.01$ s) and re-plans the CPG parameters every 1.0 s by minimising expected free energy. Grid search is excluded, as it cannot adapt online. Both methods start from the same mid-range gait, since in a no-reset bout a fatal random initialisation would end the run immediately.

Over $N = 50$ random terrains we report the fall rate, the mean velocity-tracking error $|\bar{v}_x - v^*|$, the tip-over deviation $\sqrt{(\gamma^r)^2 + (\gamma^p)^2}$, the heading (yaw) drift $|\gamma^p|$, and the forward distance, all averaged over $t > 1.5$ s.

Table I summarises the comparison. With caution equalised by the trust region, MARX-EFE falls roughly $4.5\times$ less often than online BO (12% vs. 54%) and travels

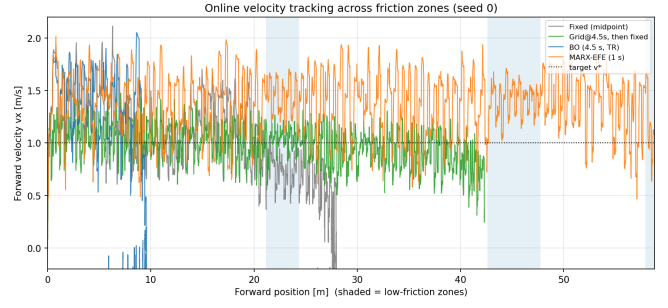


Fig. 2. Online forward-velocity tracking across friction zones on one terrain. The robot walks forward continuously (no resets); shaded bands mark low-friction zones. Both controllers track $v^* = 1.0$ m/s, but BO holds its parameters for 4.5 s while MARX-EFE re-plans every 1 s.

about 35% further, while velocity-tracking accuracy and tip-over stability are statistically indistinguishable while both remain upright (Fig. 2). Since the trust region removes BO’s large exploratory steps, the residual gap is attributable to the *update rate*: holding parameters fixed for 4.5 s, BO cannot react when it steps onto a low-friction patch, whereas MARX-EFE re-plans within 1 s and adjusts before losing balance. The advantage is therefore one of *robustness to mid-run terrain changes* rather than steady-state tracking precision.

V. DISCUSSION

VI. CONCLUSIONS

APPENDIX

ACKNOWLEDGMENT

REFERENCES

- [1] J. Collins and S. A. Richmond. Hard-wired central pattern generators for quadrupedal locomotion. *Biological Cybernetics*, 71(5):375–385, 1994.
- [2] Michael H. Dickinson, Claire T. Farley, Robert J. Full, M. A. R. Koehl, Roger Kram, and Steven Lehman. How animals move: An integrative view. *Science*, 288(5463):100–106, 2000.
- [3] Rohit Dwivedula, Sadanand Modak, Aditya Akella, Joydeep Biswas, Daehyeok Kim, and Christopher J. Rossbach. Configbot: Adaptive resource allocation for robot applications in dynamic environments. *arXiv preprint arXiv:2501.10513*, 2025.
- [4] David Eriksson, Michael Pearce, Jacob Gardner, Ryan D. Turner, and Matthias Poloczek. Scalable global optimization via local Bayesian optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019.
- [5] Martyn Goulding. Circuits controlling vertebrate locomotion: moving in a new direction. *Nature Reviews Neuroscience*, 10(7):507–518, 2009. Author manuscript; available in PMC: 2010 Mar 30.
- [6] Auke Jan Ijspeert. Special issue: Central pattern generators for locomotion control in animals and robots: A review. *Neural Networks*, 21(4):642–653, 2008.
- [7] Tim Nisslbeck and Wouter M Kouw. Coupled autoregressive active inference agents for control of multi-joint dynamical systems. In *International Workshop on Active Inference*, 2024.
- [8] Kristin Y. Pettersen. Snake robots. *Annual Reviews in Control*, 44:19–44, 2017.
- [9] Joram Soch, Thomas J. Faulkenberry, Kenneth Petrykowski, and Carsten Alfeld. *The Book of Statistical Proofs*. 2021.
- [10] Alexander Sprowitz, Ludovic Righetti, and Auke Jan Ijspeert. Passive compliant quadruped robot using central pattern generators for locomotion control. In *2008 2nd IEEE RAS & EMBS International Conference on Biomedical Robotics and Biomechanics*, pages 710–715. IEEE, 2008.